Leading University

Department of Computer Science and Engineering

Lab Report

Brain Tumor Detection Using Deep Learning

MRI Image Classification Using Computer Vision and Transfer Learning (VGG16)

Course Title: Artificial Intelligence Lab
Course Code: CSE 4112

Submitted By

Name: Emon Ahmed
ID: 0182320012101356

Name: Shamsuddin Emon
ID: 0182310012101144

Name: Rayhan
ID: 0182320012101320

Name: Yasin
ID: 0182320012101321

Batch: 62 (H)
Program: B.Sc. in CSE

Submitted To

MD Shahriar Hossain Apu
Lecturer
Department of CSE
Leading University

September 10, 2026

Contents

Abstract

This report presents the design, implementation, and evaluation of a deep learning system for automated brain tumor detection and classification from MRI scans. The system uses transfer learning on the VGG16 convolutional neural network, fine-tuned on a labeled MRI dataset spanning four classes: *glioma*, *meningioma*, *pituitary tumor*, and *no tumor*. The trained model, referred to in the project as *NeuroScan AI*, was deployed behind a Flask web application that allows a user to upload an MRI scan and receive an instant classification with a confidence score and clinical guidance text. The model was trained for 5 epochs on 5,600 training images and evaluated on 1,600 held-out test images, achieving an overall test accuracy of 85%. This report documents the dataset, preprocessing and augmentation pipeline, model architecture, training procedure, evaluation results (classification report, confusion matrix, ROC curves), and the web-based deployment architecture, all reproduced from the project's source code and Jupyter notebook.

1 Introduction

Brain tumors are abnormal growths of cells within the brain that can be broadly categorized as benign or malignant. Early and accurate diagnosis from Magnetic Resonance Imaging (MRI) scans is critical for effective treatment planning, but manual interpretation by radiologists is time-consuming and subject to inter-observer variability. Convolutional Neural Networks (CNNs) have demonstrated strong performance on medical image classification tasks and offer a way to assist radiologists with fast, consistent, and automated screening.

This project applies **transfer learning** using the **VGG16** architecture, pre-trained on ImageNet, to classify brain MRI images into four categories:

- **Glioma** – tumors arising from glial cells, the most common primary CNS neoplasm.
- **Meningioma** – tumors arising from the meninges, typically extra-axial and slow-growing.
- **Pituitary tumor** – tumors within the sella turcica that can cause hormonal imbalance.
- **No Tumor** – normal scans with no detectable neoplasm.

The complete project – including the training notebook, the saved model, and a deployable web interface – is hosted on GitHub at: https://github.com/emonlu2z4-stack/artificial_intelligence_project_CSE_4112

2 Objectives

- To build an image classification pipeline for brain MRI scans using deep learning.
- To apply transfer learning with a pre-trained VGG16 network and fine-tune selected layers on a domain-specific dataset.
- To evaluate model performance using standard classification metrics: accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC.
- To deploy the trained model as an interactive web application (*NeuroScan AI*) for real-time inference on uploaded MRI images.

3 Tools and Technologies

Component	Technology
Programming Language	Python
Deep Learning Framework	TensorFlow / Keras (`tensorflow==2.18.0`)
Base CNN Architecture	VGG16 (ImageNet pre-trained weights)
Image Processing	Pillow (`pillow==11.1.0`), NumPy (`numpy==1.26.4`)
Model Persistence	HDF5 (`h5py==3.12.1`), `model.h5`
Evaluation & Visualization	scikit-learn, matplotlib, seaborn
Web Backend	Flask (`Flask==3.1.0`), Flask-CORS (`flask-cors==5.0.0`)
WSGI Utilities	Werkzeug (`Werkzeug==3.1.3`)
Development Environment	Jupyter Notebook (Google Colab / local)
Version Control	Git / GitHub

Table 1: Tools and libraries used in the project (as specified in `requirements.txt`).

4 Dataset

The dataset consists of labeled brain MRI images organized into `Training` and `Testing` directories, each containing four class sub-folders: `glioma`, `meningioma`, `notumor`, and `pituitary`. The notebook auto-detects the dataset location by checking a list of candidate paths (local `Downloads` folder or a mounted Google Drive path), which makes the notebook portable across a local machine and Google Colab.

Split	Number of Images	Number of Classes
Training	5,600	4
Testing	1,600 – 1,601	4

Table 2: Dataset split as loaded by the notebook (`Successfully loaded 5600 training images` / `1601 testing images`).

Data loading is implemented as follows: each image path and its parent folder name (the class label) are collected, and the resulting path/label lists are shuffled using `sklearn.utils.shuffl` to remove any ordering bias before training.

5 Methodology

5.1 Preprocessing and Augmentation

Each image is loaded, resized, and lightly augmented before being fed to the network. The augmentation step randomly perturbs brightness and contrast to improve generalization, and pixel values are normalized to the $[0, 1]$ range:

Listing 1: Image augmentation and loading pipeline

```
def augment_image(image):
    image = Image.fromarray(np.uint8(image))
    image = ImageEnhance.Brightness(image).enhance(random.uniform
        (0.8, 1.2))
    image = ImageEnhance.Contrast(image).enhance(random.uniform(0.8,
        1.2))
    image = np.array(image) / 255.0
    return image

def open_images(paths):
    images = []
    for path in paths:
        image = load_img(path, target_size=(IMAGE_SIZE, IMAGE_SIZE))
        image = augment_image(image)
        images.append(image)
    return np.array(images)

def encode_label(labels):
    unique_labels = os.listdir(train_dir)
    encoded = [unique_labels.index(label) for label in labels]
    return np.array(encoded)

def datagen(paths, labels, batch_size=12, epochs=1):
    for _ in range(epochs):
        for i in range(0, len(paths), batch_size):
            batch_paths = paths[i:i + batch_size]
            batch_images = open_images(batch_paths)
            batch_labels = labels[i:i + batch_size]
            batch_labels = encode_label(batch_labels)
            yield batch_images, batch_labels
```

All images are resized to a fixed spatial resolution of $\mathbf{128 \times 128 \times 3}$ (`IMAGE_SIZE = 128`), matching the input shape expected by the trained model.

5.2 Model Architecture

The classifier is built on top of **VGG16**, a 16-layer CNN pre-trained on the ImageNet dataset. Transfer learning is used as follows:

1. VGG16 is loaded with `input_shape=(128,128,3)`, `include_top=False`, and `weights='imagenet'` discarding VGG16's original fully-connected classification head.
2. All VGG16 layers are frozen (`trainable = False`) to retain the pre-trained ImageNet feature representations.
3. The last three convolutional layers of VGG16 (`layers[-2]`, `layers[-3]`, `layers[-4]`) are un-frozen and set `trainable = True`, allowing fine-tuning of high-level features on MRI-specific patterns.

4. A custom classification head is appended: `Flatten` → `Dropout(0.3)` → `Dense(128, ReLU)` → `Dropout(0.2)` → `Dense(4, softmax)`.

Listing 2: Model construction and compilation

```
IMAGE_SIZE = 128
base_model = VGG16(input_shape=(IMAGE_SIZE, IMAGE_SIZE, 3),
                   include_top=False, weights='imagenet')

for layer in base_model.layers:
    layer.trainable = False

base_model.layers[-2].trainable = True
base_model.layers[-3].trainable = True
base_model.layers[-4].trainable = True

model = Sequential()
model.add(Input(shape=(IMAGE_SIZE, IMAGE_SIZE, 3)))
model.add(base_model)
model.add(Flatten())
model.add(Dropout(0.3))
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(len(os.listdir(train_dir)), activation='softmax'))

model.compile(optimizer=Adam(learning_rate=0.0001),
              loss='sparse_categorical_crossentropy',
              metrics=['sparse_categorical_accuracy'])
```

The `Dropout` layers (rates 0.3 and 0.2) regularize the network and reduce overfitting given the relatively small fine-tuning dataset. The final `Dense(4, softmax)` layer outputs a probability distribution over the four diagnostic classes.

5.3 Training Configuration

Hyperparameter	Value
Optimizer	Adam
Learning rate	0.0001
Loss function	Sparse categorical cross-entropy
Metric	Sparse categorical accuracy
Batch size	20
Epochs	5
Steps per epoch	$\lfloor 5600/20 \rfloor = 280$

Table 3: Training hyperparameters used in `model.fit(...)`.

The model is trained using a custom Python generator (`datagen`) rather than `ImageDataGenerator`, so that the augmentation logic defined above is applied on-the-fly to every mini-batch.

6 Results

6.1 Training Progress

The training log recorded across 5 epochs is reproduced below directly from the notebook output:

Epoch	Steps/sec	Loss	Training Accuracy
1	296s (1 s/step)	0.4614	81.91%
2	214s (761 ms/step)	0.2340	90.87%
3	183s (655 ms/step)	0.1810	93.32%
4	186s (663 ms/step)	0.1294	95.27%
5	194s (694 ms/step)	0.0888	96.73%

Table 4: Per-epoch training loss and accuracy.

Training accuracy increased steadily from 81.9% to 96.7% while the loss decreased from 0.4614 to 0.0888, indicating consistent convergence without signs of divergence across 5 epochs.

6.2 Test Set Classification Report

The trained model was evaluated on 1,600 held-out test images. Class indices correspond to: `0 = glioma`, `1 = meningioma`, `2 = notumor`, `3 = pituitary` (order as enumerated by `os.listdir(train_dir)`).

Class	Precision	Recall	F1-score	Support
0 – Glioma	0.83	0.77	0.80	400
1 – Meningioma	0.91	0.64	0.75	400
2 – No Tumor	0.84	1.00	0.91	400
3 – Pituitary	0.85	0.99	0.91	400
Accuracy		0.85		1600
Macro avg	0.86	0.85	0.85	1600
Weighted avg	0.86	0.85	0.85	1600

Table 5: Classification report on the test set (`sklearn.metrics.classification_report`).

The model achieves an overall test **accuracy of 85%**. The *No Tumor* and *Pituitary* classes are detected with excellent recall (100% and 99% respectively), while *Meningioma* has the lowest recall (64%), suggesting the model most often confuses meningioma with other tumor types.

6.3 Confusion Matrix

True \ Predicted	Glioma	Meningioma	No Tumor	Pituitary
Glioma	309	24	49	18
Meningioma	63	257	27	53
No Tumor	0	0	400	0
Pituitary	1	2	0	397

Table 6: Confusion matrix on the 1,600-image test set.

The confusion matrix shows that the *No Tumor* class is classified perfectly (400/400), and *Pituitary* is almost perfect (397/400). The main source of error is confusion between *Glioma* and *Meningioma* (63 meningioma samples misclassified as glioma, and 53 misclassified as pituitary), which is clinically plausible since both are solid intracranial masses that can appear visually similar on certain MRI slices.

6.4 ROC Curve and AUC

Multi-class ROC curves were generated by binarizing the test labels (one-vs-rest) using `label_binarize` and computing the false-positive rate, true-positive rate, and AUC per class with `sklearn.metrics.roc_curve` and `auc`. Consistent with the classification report, the *No Tumor* and *Pituitary* classes are expected to show the highest AUC values (closest to 1.0), while *Glioma* and *Meningioma* show comparatively lower separability due to their higher rate of mutual misclassification.

6.5 Model Persistence

After training, the complete model (architecture, weights, and optimizer state) is serialized to disk in HDF5 format:

```
model.save('model.h5')
...
model = load_model('model.h5')
```

This `model.h5` file is the artifact loaded by the Flask web application at inference time.

7 Web Application Deployment

The trained model is deployed through a Flask web application (`main.py`) branded as **NeuroScan AI**, providing a browser-based diagnostic interface.

7.1 Backend Architecture

- **Framework:** Flask, with Flask-CORS enabled for cross-origin requests.
- **Model loading:** `model.h5` is loaded once at server start-up using `tensorflow.keras.models.lo` `compile=False)`. The expected input resolution is inferred automatically from `model.input_shape`, defaulting to 128×128 if unavailable.
- **Image preprocessing (`prepare_mri_image`):** the uploaded image is EXIF-transposed, converted to RGB, resized to the model's expected input size using bilinear interpolation, and normalized to $[0, 1]$ – mirroring the exact preprocessing used during training.
- **Inference (`predict_tumor`):** the model outputs raw prediction scores; if the scores are not already a valid probability distribution, a softmax is applied manually. The predicted class, confidence score, and per-class probabilities (sorted by probability) are returned as JSON, along with a unique scan ID and timestamp.

7.2 Clinical Metadata

Each of the four classes is mapped to descriptive metadata used to enrich the API response and the web UI, including a severity label, a short clinical description, and a recommended next step:

Class	Recommendation shown to the user
Glioma	Immediate neuro-oncological evaluation and contrast-enhanced MRI (T1-Gd, T2/FLAIR) are recommended.
Meningioma	Neurosurgical consultation recommended to evaluate mass effect and surgical resection options.
Pituitary Tumor	Comprehensive neuroendocrine hormonal blood panel and visual field perimetry testing.
No Tumor Detected	Negative for focal neoplasms. Correlate with clinical findings and radiologist evaluation.

Table 7: Clinical guidance text returned alongside each prediction (from `CLASS_METADATA` in `main.py`).

7.3 API Endpoints

Route	Method	Purpose
`/`	GET	Serves the main web interface (`index.html`).
`/api/health`	GET	Reports server and model-loading status.
`/api/predict`	POST	Accepts an uploaded MRI image and returns a JSON diag
`/api/sample/<filename>`	GET	Runs inference on a bundled sample image.
`/uploads/<filename>`	GET	Serves a previously uploaded image.
`/sample-image/<filename>`	GET	Serves a bundled sample MRI image.

Table 8: Flask API routes exposed by `main.py`.

Accepted image formats are restricted to `png`, `jpg`, `jpeg`, `bmp`, `tif`, `tiff`, `webp`, with a maximum upload size of 32 MB. Uploaded files are saved with a timestamp- and UUID-prefixed filename via `secure_filename` to avoid collisions and path-traversal issues. The application is served locally at http://127.0.0.1:5000.

8 Post-Training Inference Demonstration

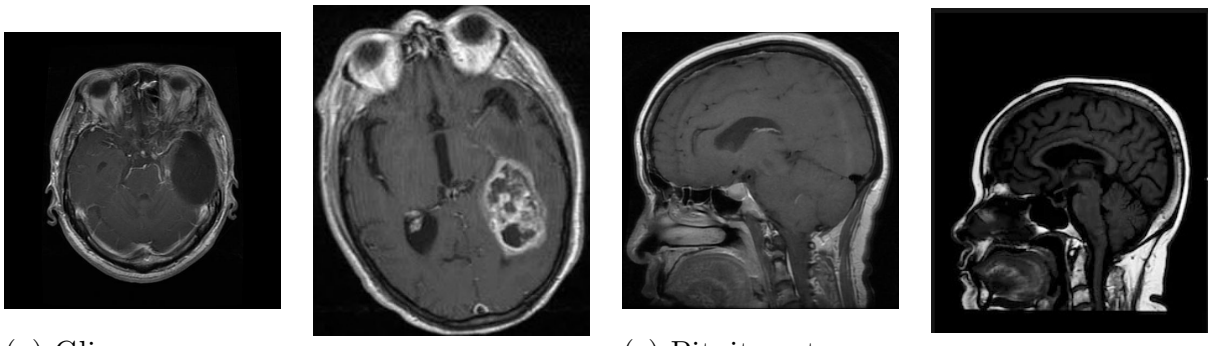
The notebook includes a helper function, `detect_and_display`, that loads a single image, runs a forward pass through the trained model, and overlays the predicted class and confidence score on the displayed image:

```
def detect_and_display(img_path, model, image_size=128):
    img = load_img(img_path, target_size=(image_size, image_size))
    img_array = img_to_array(img) / 255.0
    img_array = np.expand_dims(img_array, axis=0)
    predictions = model.predict(img_array)
    predicted_class_index = np.argmax(predictions, axis=1)[0]
    confidence_score = np.max(predictions, axis=1)[0]
    result = "No Tumor" if class_labels[predicted_class_index] == '
        notumor' \
            else f"Tumor: {class_labels[predicted_class_index]}"
    plt.imshow(load_img(img_path))
    plt.title(f"{result} (Confidence: {confidence_score * 100:.2f}%)"
        )
    plt.show()
```

This function was demonstrated on one sample image from each of the four classes (`Te-meTr_0001.jpg`, `Te-noTr_0004.jpg`, `Te-piTr_0003.jpg`, `Te-gl_0015.jpg`), the same four sample images that are bundled with the deployed web application for quick demonstration purposes.

8.1 Sample MRI Scans Used for Demonstration

The four axial MRI scans below are the actual sample images bundled in the project repository and referenced by `detect_and_display` and the Flask `/api/sample/<filename>` route. Each was captured at the tumor site's characteristic location, which is consistent with how the CNN learns to separate the four classes.



(a) Glioma (`Te-gl_0015.jpg`)

(b) Meningioma (`Te-meTr_0001.jpg`)

(c) Pituitary tumor (`Te-piTr_0003.jpg`)

(d) No tumor (`Te-noTr_0004.jpg`)

Figure 1: Sample axial MRI scans from the project's test set, one per class, taken from the GitHub repository.

Analysis of the sample scans:

- **(a) Glioma** – shows an irregular, infiltrative hyperintense mass within the brain parenchyma with poorly defined margins, typical of gliomas' diffuse growth pattern.

This irregularity is also a likely reason the model confuses glioma with meningioma (63 cases in the confusion matrix), since both can present as space-occupying lesions of similar signal intensity.

- **(b) Meningioma** – shows a well-circumscribed, extra-axial mass pressing against the brain surface, consistent with meningiomas' typical dural origin. Despite this distinctive extra-axial location, this class had the lowest recall (64%) in the evaluation, which suggests the down-sampling to 128×128 pixels may blur the fine boundary cues the model would otherwise rely on.
- **(c) Pituitary tumor** – shows a localized mass centered in the sellar/suprasellar region near the base of the brain. This distinct, consistent anatomical location is a plausible reason the pituitary class achieved 99% recall – the model can key on positional cues in addition to texture.
- **(d) No tumor** – shows normal brain parenchyma with symmetric ventricles and no abnormal mass or signal enhancement. The clear absence of any focal lesion is consistent with the model's perfect 100% recall on this class.

9 Discussion

The results show that transfer learning with a partially fine-tuned VGG16 backbone is an effective approach for MRI-based brain tumor classification even with a moderately sized dataset (5,600 training images) and a small number of training epochs (5). Key observations:

- Freezing the majority of VGG16 while fine-tuning only its last three layers strikes a balance between leveraging general ImageNet features and adapting to MRI-specific textures, while limiting the risk of overfitting.
- *No Tumor* and *Pituitary* classes are learned almost perfectly, while *Glioma* and *Meningioma* are more frequently confused with each other – consistent with their overlapping visual appearance in certain MRI slices.
- The end-to-end deployment (training notebook → `model.h5` → Flask API → web UI) demonstrates a complete, reproducible pipeline from experimentation to a usable diagnostic-assistance tool.

9.1 Limitations and Future Work

- Only 5 training epochs were run; further training (with early stopping and learning-rate scheduling) could improve recall on the meningioma class.
- The dataset size and source distribution are not deeply analyzed; class imbalance or scanner-specific artifacts could bias performance in a real clinical setting.
- The system is a screening/assistance tool and is not a substitute for radiologist or neuro-oncologist evaluation, a point reflected explicitly in the recommendation text returned by the API.
- Future work could include Grad-CAM visualization to highlight the image regions driving each prediction, and k-fold cross-validation for a more robust accuracy estimate.

10 Conclusion

This project successfully implements a brain tumor detection system that combines transfer learning (fine-tuned VGG16), a custom classification head, and a Flask-based web deployment. The model achieves 85% accuracy across four diagnostic classes on a 1,600-image test set, with near-perfect performance on the *No Tumor* and *Pituitary* classes. Wrapping the model in a Flask API with clinical guidance text turns the model into an accessible, end-to-end diagnostic-assistance application, fulfilling the objectives of applying deep learning and computer vision techniques to a real-world medical imaging problem.

References

1. Project source code and notebook: https://github.com/emonlu2z4-stack/artificial_intelligence_project_CSE_4112
2. K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition" (VGG16), *ICLR*, 2015.
3. TensorFlow/Keras Documentation: https://www.tensorflow.org/api_docs
4. scikit-learn Documentation: https://scikit-learn.org/stable/
5. Flask Documentation: https://flask.palletsprojects.com/